Progressive Web App & Web Communication Technique

IF3110 Web-Based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Motivation

- **What Users Expect Today?**
  - **Installation**: an icon on the home screen or desktop
  - **Fast loading**: the app opens instantly, even on slow networks
  - **Unreliable connectivity**: it still works on a train, in a basement, or on a weak campus Wi-Fi
  - **Timely updates**: new information appears without pressing refresh

What is a Progressive Web App ?

- Experiences that combine the best of web with the best of app and becomes useful to users from the very first visit in a tab.

- “A PWA is a web application that camouflage itself as a native mobile application with a home screen app icon, responsive design, fast load speed, offline functionality, and more”
- https://clevertap.com/blog/progressive-web-apps/



Objectives

- Reliable - Load instantly and never show the downasaur, even in uncertain network conditions.
- Fast - Respond quickly to user interactions with silky smooth animations and no janky scrolling.
- Engaging - Feel like a natural app on the device, with an immersive user experience.

Key Characteristics of PWA

- Progressive - Works for every user regardless of browser and users
- Responsive - Fits any form factor; desktop,mobile,tablet etc
- Connectivity Independent - Enhanced with Service Workers to Work Offline or on Lie-Fi.
- Fresh - Always up to date courtesy of service worker update process

Key Characteristics of PWA - cont'd

- App Like - Feels like an app to the user because it's built on the app-shell model
- Re-engageable - Makes re-engagement easy through features like push notifications.
- Installable - Allows user to 'keep' apps on their homescreen without the hassle of an app store.
- Safe - Served via HTTPS to prevent snooping and ensure content isn't tampered with.

Web App Manifest

- is a simple JSON file
- What it does
  - Creating a manifest and linking it to your page are straightforward processes.
  - Control what the user sees when launching from the home screen.
  - This includes things like a splash screen, theme colors, and even the URL that's opened.

Web App Manifest – cont'd

- Web app manifests provide the ability to save a site bookmark to a device's home screen. When a site is launched this way:
  - It has a unique icon and name so that users can distinguish it from other other sites.
  - It displays something to the user while resources are downloaded or restored from cache.
  - It provides default display characteristics to the browser to avoid too abrupt a transition when site resources become available.
  - It does all this through the simple mechanism of metadata in a text file. That's the web app manifest.

Installable - Manifest.webmanifest

- User can Add App to Homescreen
- Easy access to Web App from homescreen
- Add a Manifest.webmanifest file to your app's directory
- Add this to your index.html to tell the browser about your manifest location:

```
<link rel="manifest
href="/manifest.webmanifest">
```

```
{
  "name": "Classroom App",
  "short_name": "Classroom",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "theme_color": "#1e40af",
  "background_color": "#ffffff",
  "icons": [
    { "src": "/icons/192.png", "sizes":
"192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes":
"512x512", "type": "image/png" }
  ]
}
```

Read more : https://www.w3.org/TR/2016/WD-appmanifest-20160331/

Installable - Manifest.webmanifest

- User can Add App to Homescreen
- Easy access to Web App from homescreen
- Add a Manifest.webmanifest file to your app's directory
- Add this to your index.html to tell the browser about your manifest location:

```
<link rel="manifest
href="/manifest.webmanifest">
```

```
{
  "name": "Classroom App",
  "short_name": "Classroom",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "theme_color": "#1e40af",
  "background_color": "#ffffff",
  "icons": [
    { "src": "/icons/192.png", "sizes":
"192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes":
"512x512", "type": "image/png" }
  ]
}
```

Read more : https://www.w3.org/TR/2016/WD-appmanifest-20160331/

Installable - Manifest.webmanifest

- User can Add App to Homescreen
- Easy access to Web App from homescreen
- Add a Manifest.webmanifest file to your app's directory
- Add this to your index.html to tell the browser about your manifest location:

```
<link rel="manifest
href="/manifest.webmanifest">
```

```
{
  "name": "Classroom App",
  "short_name": "Classroom",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "theme_color": "#1e40af",
  "background_color": "#ffffff",
  "icons": [
    { "src": "/icons/192.png", "sizes":
"192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes":
"512x512", "type": "image/png" }
  ]
}
```

Read more : https://www.w3.org/TR/2016/WD-appmanifest-20160331/

Display Mode

Mode	What the user sees	Good for
fullscreen	Entire screen, no browser UI, no status bar	Games, video, kiosks
standalone	Own window, like a native app, no address bar	Most PWAs
minimal-ui	Own window with a few navigation controls (back, reload)	Apps where users need back/reload
browser	A normal browser tab (default)	Regular websites

Display Mode



Source: https://firt.dev/pwa-2020/

Connectivity Independent

- **What is a Service Worker ?**
  A script run by the **browser** in the **background**, separate from a webpage, that opens the door to features which don't need a web page or user interaction
- **Note**
  - Can't access the DOM directly. Only way to communicate with pages is through postMessage.
  - Service worker can "hijack" network requests, giving you the developer absolute control.
  - Terminated when idle. Restarted needed.
  - Persist data using IndexedDb or something similar[1]
  - Can make request and cache for better performance and lower network usages
  - Service workers make extensive use of promises[2]
- **Read more :**
  - http://www.html5rocks.com/en/tutorials/service-worker/introduction/
  - http://www.html5rocks.com/tutorials/es6/promises/

Service Worker

- Client-side proxy in JavaScript
- Net requests, but also push & sync

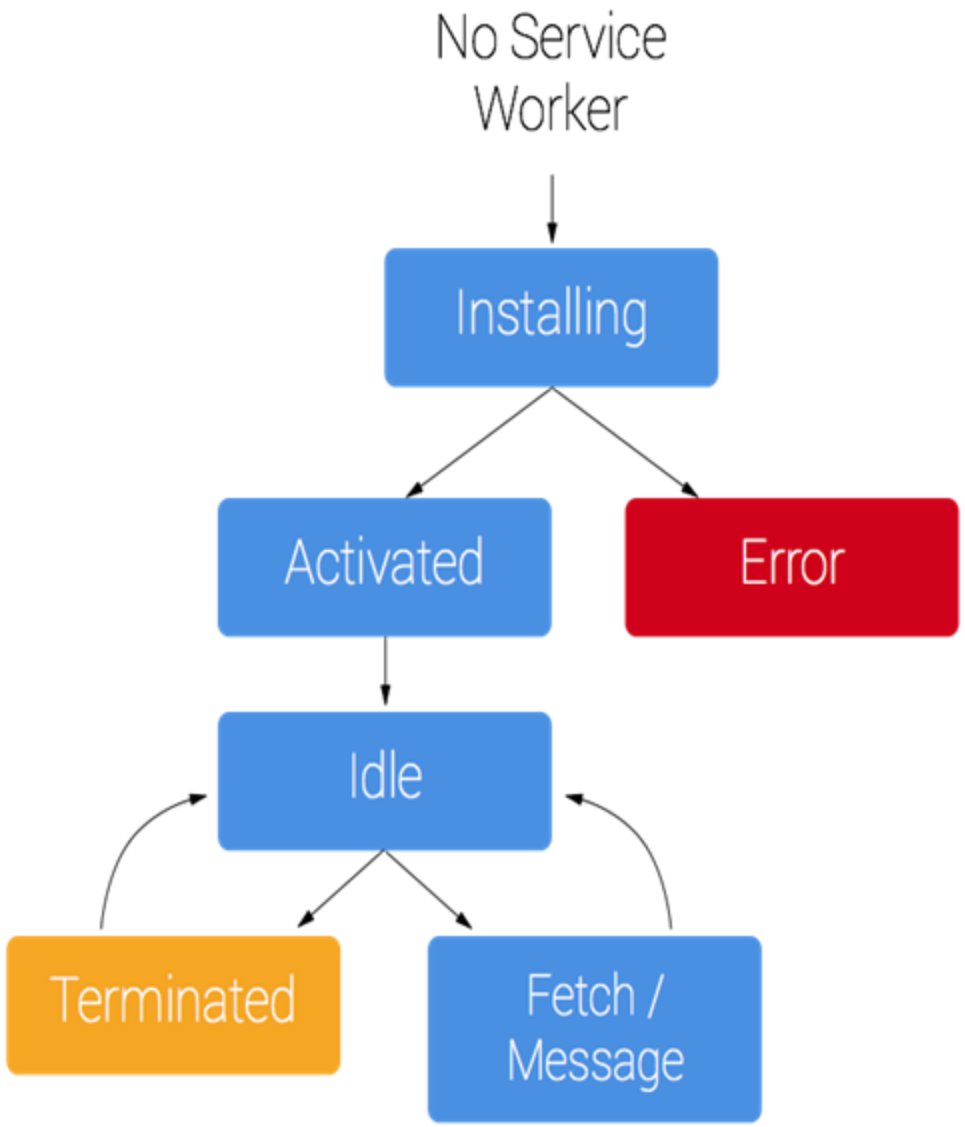


Regstration

- In your page, you can add this script to register service worker:

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', async () => {
    try {
      const reg = await
navigator.serviceWorker.register('/sw.js');
      console.log('SW registered, scope:', reg.scope);
    } catch (err) {
      console.error('SW registration failed:', err);
    }
  });
}
```

Service Workers - LifeCycle



Lifecycle Listener

```
self.addEventListener('install', event => {
  console.log('V1 installing…');

  // cache a cat SVG
  event.waitUntil(
    caches.open('static-v1').then(cache => cache.add('/cat.svg'))
  );
});

self.addEventListener('activate', event => {
  console.log('V1 now ready to handle fetches!');
});


// Intercepting request
self.addEventListener('fetch', event => {
  const url = new URL(event.request.url);

  // serve the cat SVG from the cache if the request is
  // same-origin and the path is '/dog.svg'
  if (url.origin == location.origin && url.pathname == '/dog.svg') {
    event.respondWith(caches.match('/cat.svg'));
  }
});
```

Cache API

- **Previously:** The Cache API stores network requests and their corresponding responses for later reuse.
- **Where can we use it?** It is available in both the web page and a **service worker**, through `caches`.
- **Inside a service worker:** We can intercept requests using the `fetch` event and decide whether to return a cached response or retrieve one from the network.
- **Why is this useful?** Cached HTML, CSS, JavaScript, and images can help the application load faster and remain usable offline.

```
// sw.js
self.addEventListener('fetch', (event) => {
  const request = event.request;

  // Only cache GET requests.
  if (request.method !== 'GET') return;

  event.respondWith((async () => {
    const cache = await caches.open('static-v1');
    const cached = await cache.match(request);

    if (cached) return cached;

    const response = await fetch(request);

    if (response.ok) {
      await cache.put(request, response.clone());
    }

    return response;
  })());
});
```

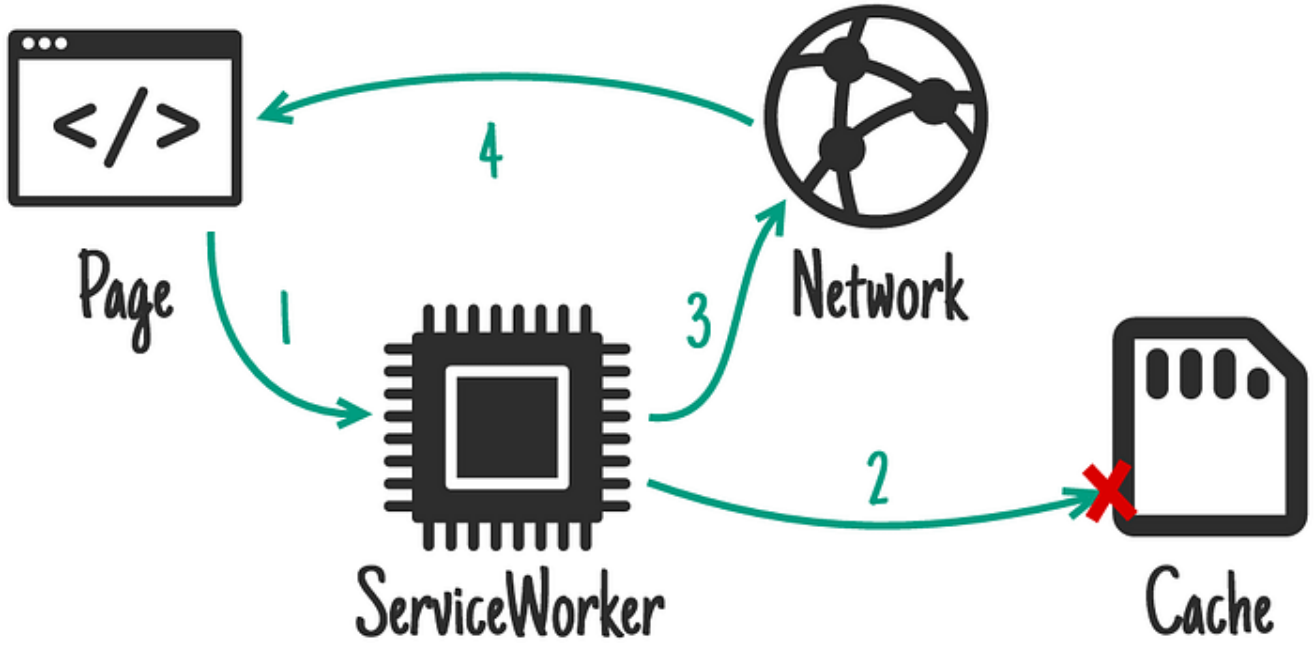
Caching Strategies - Responding to request

- Cache Only
- Network Only
- Cache, falling back to Network
- Network, falling back To Cache
- Stale While Revalidate
- Cache & Network Race
- Cache then network
- Generic Fallback
- SW Push Notification Flow

- Read More : https://jakearchibald.com/2014/offline-cookbook/
- https://jakearchibald.com/2016/caching-best-practices/

Cache First, Network Fallback

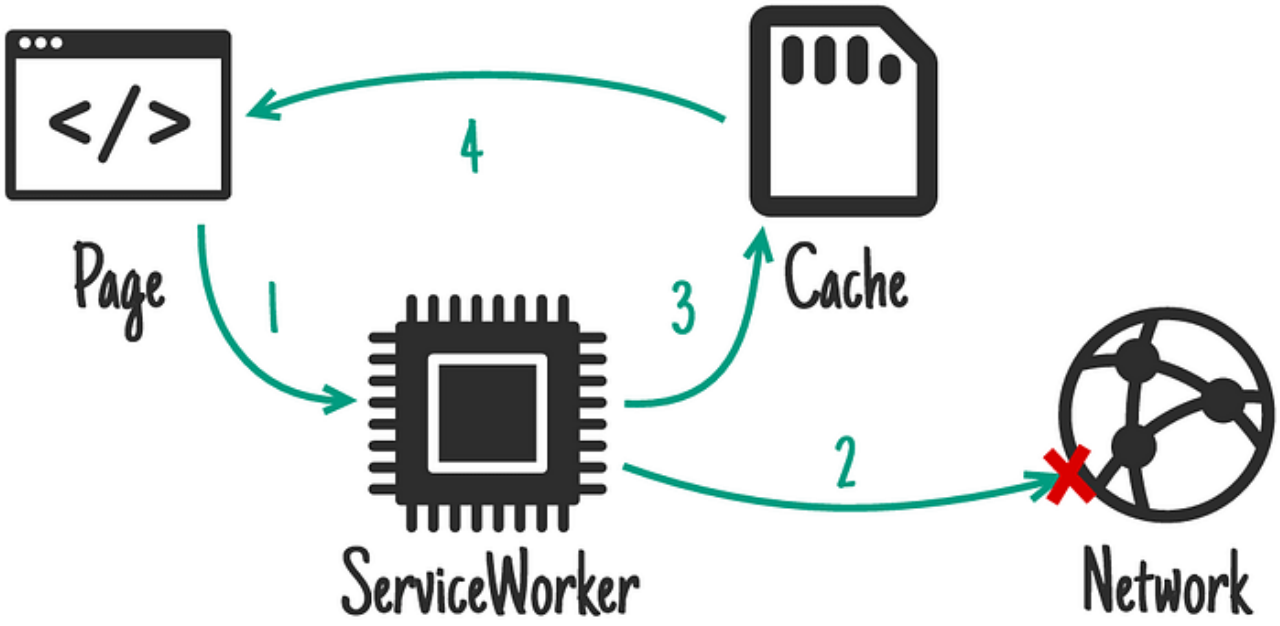
- The service worker will loads the local (cached) assets (HTML, CSS, images, fonts, etc).
- If cached content is not available, **service worker returns a response from the web** and **caches the network response**.
- **Best for** resource that very unlikely change, such as static image.



https://medium.com/animall-engineering/best-caching-strategies-progressive-web-app-pwa-c610d65b2009

Network First, Cache Fallback

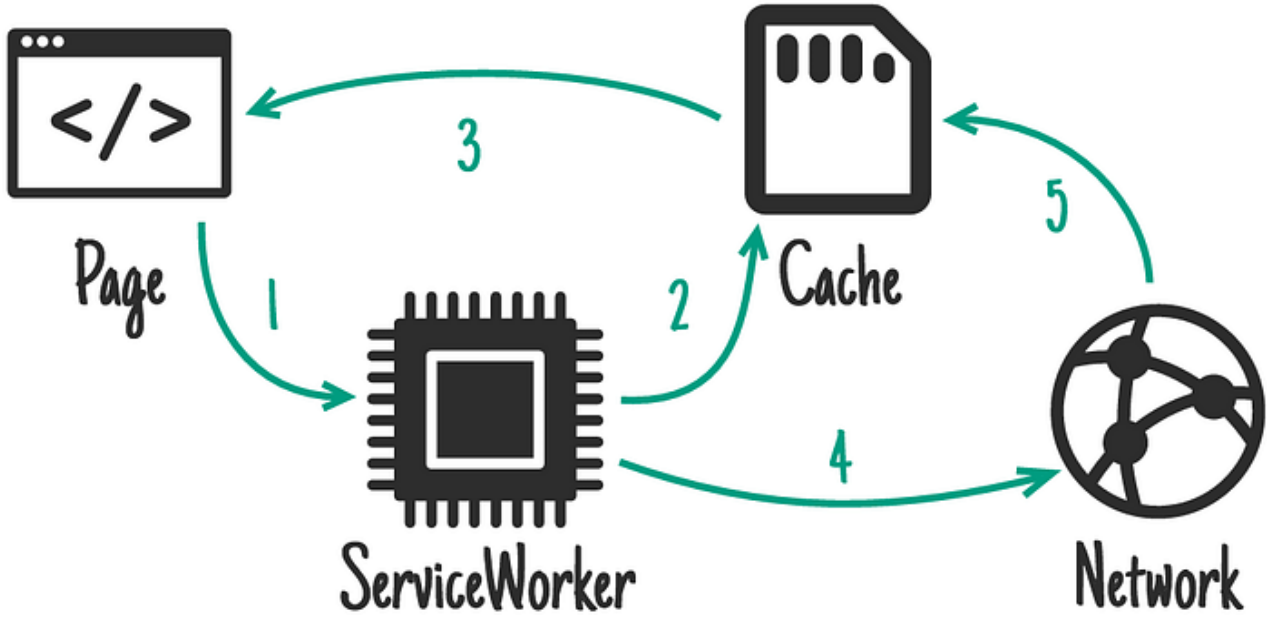
- The service worker will check the network first for a response and, if successful, returns current data to the page.
- If the network request fails, then the service worker returns the cached entry instead.
- **Best for** resource that **being up to date matters more than loading instantly**, but showing an older copy is still **better than an error when the network is down.**



https://medium.com/animall-engineering/best-caching-strategies-progressive-web-app-pwa-c610d65b2009

Stale with Revalidate

- In this strategy, The request will be responded as quick ad possible with cached response.
- The network connection will be used to update cache.
- If it's not cached, it will fallback to network request.
- This is a fairly common strategy where having the most up-to-date resource is not vital to the application.

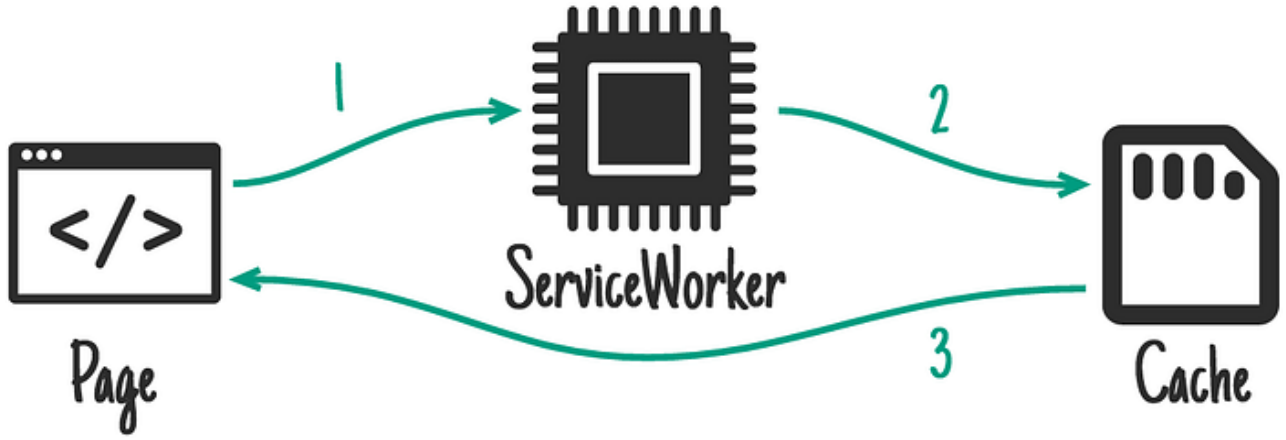


https://medium.com/animall-engineering/best-caching-strategies-progressive-web-app-pwa-c610d65b2009

Cache Only

- Requests are answered only from the cache. The service worker never falls back to the network.
- Required resources are pre-cached during the service worker's install event.
- Suitable for static resources that are bundled with a particular application version, such as an offline page, icons, CSS, and JavaScript.

- **Warning:**
  - If a resource is missing from the cache, the request fails—even when the user is online.
  - Updated resources must be explicitly added to the cache; server changes are not fetched automatically.



https://medium.com/animall-engineering/best-caching-strategies-progressive-web-app-pwa-c610d65b2009

Offline Support

- **Offline support** allows a web application to remain useful when the network is unavailable or unreliable.

- Technique
  - **Service worker:** Intercept requests and apply a strategy for obtaining a response.
  - **Cache API:** Store and retrieve resources such as HTML, CSS, JavaScript, images, and selected responses.
  - **Pre-caching:** Store essential resources during service worker installation.
  - **Runtime caching:** Store additional resources as users access them.
  - **IndexedDB:** Save structured application data, such as learning materials, drafts, and pending changes.
  - **Synchronization:** Retry pending changes after reconnecting. Use Background Sync where supported, with a retry mechanism in the application as a fallback.

Workbox

- Workbox is a set of JavaScript libraries that simplifies caching and request handling in service workers, helping web apps load faster and work offline.
- **Capabilities**
  - **Precaching:** Saves essential app files during installation for offline access.
  - **Runtime caching:** Stores responses as users browse, such as images and API data.
  - **Caching strategies:** Controls whether requests use cached content, the network, or both.
  - **Background sync:** Queues failed requests and retries them later.

Web Communication Tehnique

Why do We need Other Communication Technique?

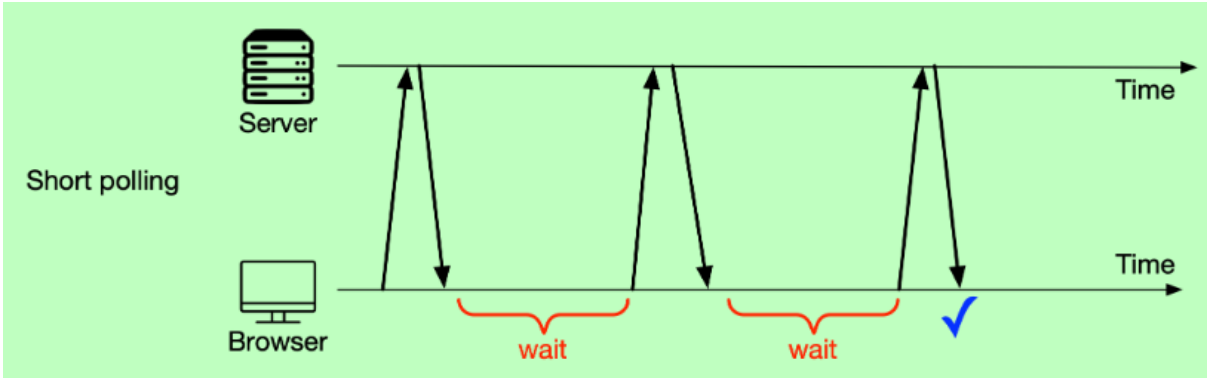
- HTTP is normally **client-initiated**: the browser asks, the server answers
- Different interactions need different communication techniques:
  - **Announcements and live scores:** Receive updates from the server.
  - **Chat and collaboration:** Exchange messages in both directions.
  - **Video calls:** Exchange live audio and video.
- The server **cannot simply push** a new announcement to a page that didn't ask
- Without something extra, users must **refresh** to see updates

Communication Technique

Technique	Communication direction	Update delivery	Works with tab closed?	Best suited for
Short polling	Client requests -> server responds	On the next periodic check	No	Dashboards, occasional status checks
Long polling	Client requests -> server waits to respond	As data becomes available; reconnects after each response	No	Infrequent updates, compatibility fallbacks
SSE	Server -> client	Continuous event stream	No	Live feeds, progress updates, AI text streaming
WebSocket	Client <-> server	Live messages over a persistent connection	No	Chat, collaborative editing, multiplayer games
Web Push	Server -> browser via push service	Event-driven; delivery timing can vary	Yes*	Notifications, reminders, re-engagement
WebRTC	Peer to peer connection, with relay if needed	Live media or data	No	Video calls, screen sharing, peer data transfer

Short Polling

- The browser repeatedly requests updates, for example every five seconds.
- The server responds immediately with new records or an empty result.
- The browser updates the UI, waits, and sends another request.



https://bytebytego.com/guides/shortlong-polling-sse-websocket/

Short Polling

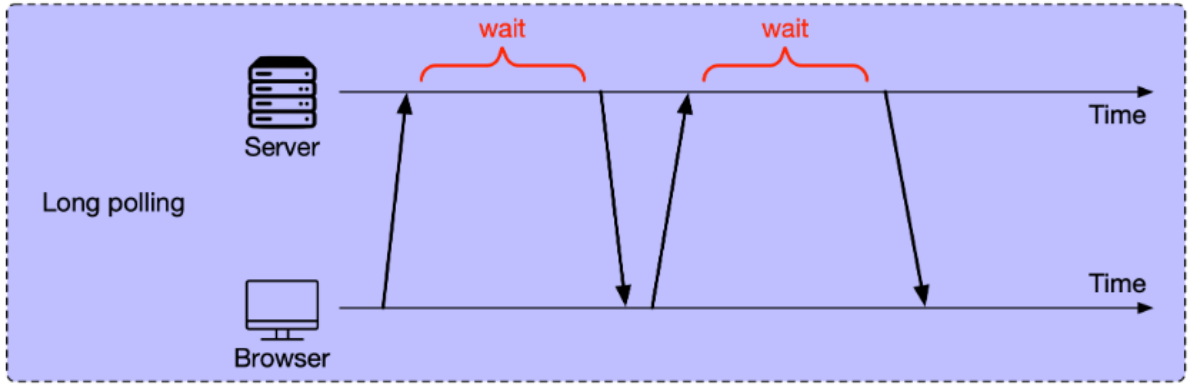
- **Suitable use cases**
  - **Periodic updates:** Announcements, dashboards, and order status checks.
  - **Infrequently changing data:** Features where a few seconds of delay are acceptable.
  - **Simple applications:** Reuse existing HTTP endpoints without maintaining waiting requests or streams.
- **Trade-off**
  - **Update delay:** New data remains unseen until the next request.
  - **Unnecessary requests:** The client keeps checking even when nothing has changed.
  - **High server load:** Frequent polling increases traffic and processing costs.
  - **Overlapping requests:** Requests can accumulate if responses take longer than the polling interval.

Short Polling Example

```
let lastID = 0;
const controller = new AbortController();
async function poll() {
  try {
    const r = await fetch(`/announcements?after=${lastID}`, {
      signal: controller.signal, cache: 'no-store'
    });
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    for (const item of await r.json()) {
      renderAnnouncement(item);
      lastID = item.id;
    }
  } catch (e) {
    if (e.name !== 'AbortError') console.error(e);
  }
  if (!controller.signal.aborted) setTimeout(poll, 5000);
}
poll();
```

Long Polling

- The client requests records after its last received ID.
- If records exist, the server responds immediately.
- Otherwise, the server **holds the request** until data arrives or a timeout expires.
- After receiving a response, the client immediately starts another request.
- **Suitable for:** Event delivery through ordinary HTTP when a streaming approach is not appropriate.
- **Trade-off:**
  - Each waiting client consumes connection and server resources.
  - Proxies and load balancers may enforce timeouts. The server's waiting period should be shorter than these limits.



https://bytebytego.com/guides/shortlong-polling-sse-websocket/

Long Polling

- **Suitable for:**
  - **Infrequent updates:** Announcements, alerts, and new task assignments.
  - **Waiting for an event:** Background job completion or order status changes.
  - **Compatibility fallback:** Applications where persistent streaming is unavailable.
- **Trade-off:**
  - Each waiting client consumes connection and server resources.
  - Proxies and load balancers may enforce timeouts. The server's waiting period should be shorter than these limits.

Long Polling Example: Javascript Client

```
let lastID = 0;
const controller = new AbortController();
const delay = ms => new Promise(resolve => setTimeout(resolve, ms));
async function listen() {
  while (!controller.signal.aborted) {
    try {
      const r = await fetch(`/announcements/wait?after=${lastID}`, {
        signal: controller.signal, cache: 'no-store'
      });
      if (!r.ok) throw new Error(`HTTP ${r.status}`);
      for (const item of await r.json()) {
        renderAnnouncement(item);
        lastID = item.id;
      }
    } catch (e) {
      if (controller.signal.aborted) break;
      await delay(3000);
    }
  }
}
listen();
```

Long Polling Example: Go Server

```
func announcementWait(w http.ResponseWriter, r *http.Request) {
    timer := time.NewTimer(25 * time.Second)
    defer timer.Stop()

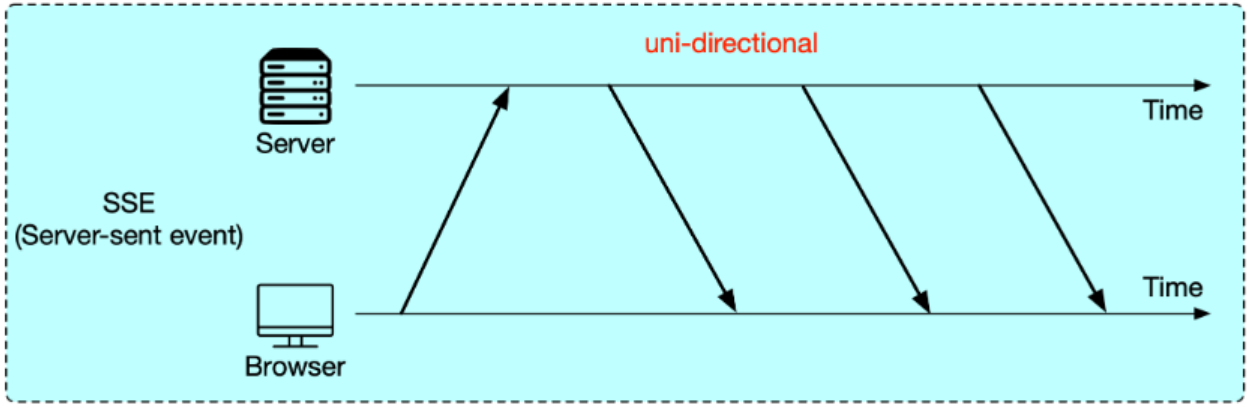
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Cache-Control", "no-store")

    // Subscribe returns a separate channel for this client.
    updates, unsubscribe := subscribe()
    defer unsubscribe()

    select {
    case message := <-updates:
        json.NewEncoder(w).Encode(message)
    case <-timer.C:
        json.NewEncoder(w).Encode(nil) // Timeout: return null.
    case <-r.Context().Done():
        return // Client disconnected.
    }
}
```

Server-Sent Event

- **Server-Sent Events** sends updates through an HTTP response that remains open.
- The server writes another event whenever new data is available.
- The browser receives the stream through **EventSource**.
- The stream carries updates **from server to client**.
- Client actions can use separate HTTP requests, such as POST /announcements.



https://bytebytego.com/guides/shortlong-polling-sse-websocket/

Server-Sent Event

- **Suitable use cases**
  - **Live feeds:** Announcements, news, and activity updates.
  - **Progress updates:** Background jobs, file processing, and report generation.
  - **Monitoring dashboards:** Live metrics, status changes, and logs.
  - **Streaming text:** Display generated text incrementally.
- **Trade-offs**
  - **One-way communication:** Client-to-server actions require separate HTTP requests.
  - **Text-based format:** Binary data requires encoding or another transport.
  - **Persistent connections:** Consume server resources and require appropriate proxy buffering and timeout settings.
  - **Recovery needs implementation:** Event IDs help track progress, but the server must store and replay missed events.

SSE HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

```
event: announcement
id: 1
data: {”data": ”New Assignment", "timestamp":
1718872000}
```

<- after a few seconds

SSE: JS Client

```
const source = new EventSource('/events');
source.addEventListener('announcement', event => {
  try {
    const item = JSON.parse(event.data);
    renderAnnouncement(item);
  } catch (error) {
    console.error('Invalid event', error);
  }
});

source.onopen = () => console.log('Feed connected');
source.onerror = () => console.log('Feed interrupted');
// Call source.close() when leaving this feature.
```

SSE: Go Server

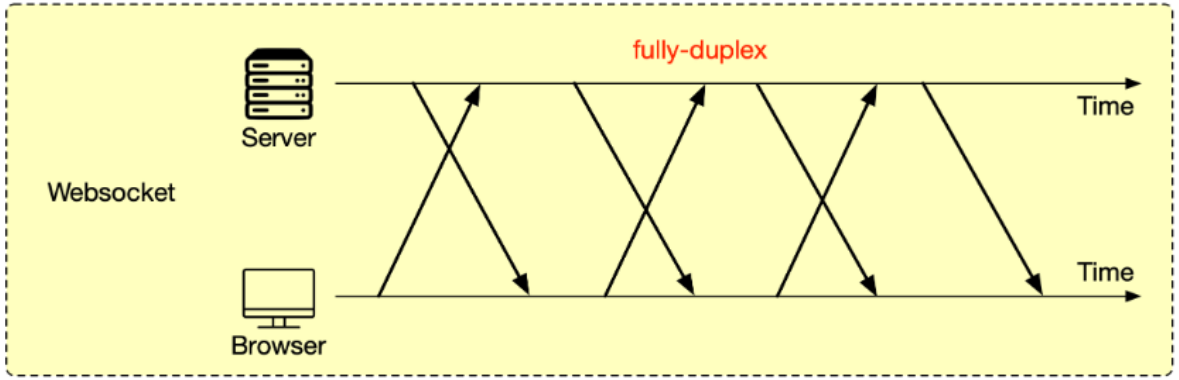
```
func eventHandler(w http.ResponseWriter, r *http.Request) {
    events, unsubscribe := subscribe()
    defer unsubscribe()

    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    rc := http.NewResponseController(w)
    if err := rc.Flush(); err != nil { return }

    for {
        select {
        case item, ok := <-events:
            if !ok { return }
            data, err := json.Marshal(item)
            if err != nil { return }
            if _, err := fmt.Fprintf(w,
                "event: announcement\ndata: %s\n\n", data); err != nil {
                return
            }
            if err := rc.Flush(); err != nil { return }
        case <-r.Context().Done():
            return
        }
    }
}
```

WebSocket

- WebSocket provides a persistent **bidirectional** connection.
- Both browser and server can send messages independently after connection setup.
- Messages can contain text or binary data.
- The application defines message types and their meaning.



https://bytebytego.com/guides/shortlong-polling-sse-websocket/

WebSocket

- **Suitable use cases**
  - **Chat applications:** Exchange messages and typing indicators.
  - **Collaborative editing:** Share document changes and cursor positions.
  - **Multiplayer games:** Exchange player actions and game state.
  - **Interactive dashboards:** Receive live updates and send user commands.
- **Trade-offs**
  - **Connection management:** Requires reconnection logic, heartbeats, and cleanup.
  - **Resource usage:** Persistent connections and queued messages consume resources.
  - **Recovery complexity:** Missed messages require application-level storage and replay.
  - **Scaling complexity:** Multiple servers must coordinate delivery to connected users.
  - **Slow clients:** Unbounded message queues can exhaust memory.

WS: JS Client

```
const scheme = location.protocol === 'https:' ? 'wss' : 'ws';
const socket = new WebSocket(`${scheme}://${location.host}/chat`);
socket.onopen = () => console.log('Chat connected');
socket.onmessage = event => {
  const message = JSON.parse(event.data);
  renderMessage(message);
};

socket.onerror = () => console.log('Connection error');
socket.onclose = () => console.log('Chat disconnected');
function sendChat(text) {
  if (socket.readyState !== WebSocket.OPEN) return false;
  socket.send(JSON.stringify({ type: 'chat', text }));
  return true;
}
```

WS: Go Server

```
import (
    "net/http"
    "strings"

    "github.com/coder/websocket"
    "github.com/coder/websocket/wsjson"
)

func chat(w http.ResponseWriter, r *http.Request) {
    c, err := websocket.Accept(w, r, nil)
    if err != nil {
        return
    }
    defer c.CloseNow()
    c.SetReadLimit(4096)

    client := room.Join(c)
    defer room.Leave(client)

    for {
        var msg Message
        if err := wsjson.Read(r.Context(), c, &msg); err != nil {
            return
        }
        if msg.Type != "chat" || strings.TrimSpace(msg.Text) == "" {
            continue
        }
        room.Publish(client, msg)
    }
}
```

Why Web Push Notifications?

- Polling, SSE, and WebSocket keep information updated **while the application is running**.
- **What happens when the user closes the page?** The page can no longer poll the server or maintain its connections.
- Some events still deserve the user's attention, such as a new message, an order update, or an important reminder.
- **Web Push** allows the server to reach a service worker even when the page is not open, subject to browser support and user permission.
- The service worker can display a notification that invites the user back into the application.

Web Push Notifications

- **Web Push** delivers server messages to a service worker, even when the application's page is not open.
- It uses two complementary APIs:
  - **Push API:** Receives messages from a push service.
  - **Notifications API:** Displays notifications to the user.
- This technique allow users to opt-in to timely updates from sites they love and allow you to effectively re-engage them with customized, relevant content

Firebase Cloud Messaging

- Firebase Cloud Messaging (FCM) is Google's messaging service that lets your backend send notifications to web users—even when the website's tab is closed, subject to browser and OS support.
- **How it works**
  - **Subscribe:** The user grants notification permission. Your web app registers a service worker and obtains an FCM registration token.
  - **Send:** Your backend sends a message through FCM, targeting that token.
  - **Receive:** The browser receives the push. A service worker handles background messages and displays the notification; an active page handles foreground messages.